

Lab Assignment 7

From Requirements to UML Class Design, Implementation and Execution

1 Objective

In this lab, you will take a real-world requirement and turn it into a working piece of software. That means identifying the classes involved, working out how they relate to one another, drawing a UML class diagram, implementing the classes in Java, building a command-line driver to run the system, and testing that everything behaves the way it should.

2 Problem Statement

You are building an online movie-ticket booking system.

The system manages multiple movies, each with a title, duration, language, genre, and certificate. These movies are shown across different cinemas. Each cinema has multiple screens, and each screen has multiple seats.

A single movie can have multiple shows at different times. A customer picks a movie, chooses a cinema and a show, selects available seats, and pays for the booking. Once payment succeeds, the system generates a booking along with the corresponding tickets.

If a customer cancels a booking in line with the cancellation policy, the payment may be refunded.

3 Learning Outcomes

#	By the end of this lab, you should be able to
1	Identify classes, attributes, and responsibilities from a natural-language requirement.
2	Identify relationships and multiplicities between classes.
3	Create a UML class diagram that represents your design.
4	Implement the identified classes using object-oriented programming.
5	Develop a Main / Test Driver that runs from the command line.
6	Run the required functional test cases and compare expected behavior against actual behavior.

4 Part A: Requirement Analysis

Identify the classes needed to model this system. For each class, list its attributes, its responsibilities (what it does, not just what it stores), and a short justification for why it deserves to be its own class.

Don't turn every noun in the problem statement into a class. Some things, a seat number or a ticket price, for instance, might just be attributes of another class rather than classes in their own right. Think about what actually needs its own identity, state, and behavior before you give it a class.

5 Part B: Identify Relationships

Once your classes are in place, work out how they connect to one another. For each relationship, decide whether it is an association, aggregation, composition, dependency, or inheritance, and specify the multiplicity (one-to-one, one-to-many, and so on) along with your reasoning.

Use the questions below as a starting point:

Relationship	Question to consider
Cinema and Screen	How many screens can a cinema contain?
Screen and Seat	How are physical seats associated with a screen?
Movie and Show	Can one movie have multiple shows?
Show and Cinema / Screen	Where does a particular show take place?
Customer and Booking	How many bookings can a customer make?
Booking and Ticket	When are tickets generated?
Booking and Payment	When should a booking be considered confirmed?
Payment and Refund	When can a refund be generated?

6 Part C: Design Question, Seat Availability

Here's a scenario to think through: Screen 1 in a cinema has a physical seat A1. That screen runs an Avengers show at 6:00 PM and another show at 9:00 PM. Seat A1 gets booked for the 6:00 PM show.

Should A1 automatically become unavailable for the 9:00 PM show too? Explain your reasoning, and decide whether it makes sense to introduce a separate class, something like `ShowSeat`, to track seat availability per show rather than per screen.

7 Part D: UML Class Diagram

Put together a complete UML class diagram covering every class from Part A: their attributes (with types), methods, visibility, and how they relate to one another from Part B, including multiplicities. Use standard UML notation throughout. Arrows, diamonds, and multiplicity labels should all follow convention so the diagram is easy for anyone else to read.

8 Part E: Class Implementation

Implement the classes from your UML diagram. You can use any object-oriented language, Java, Python, C++, C#, and Kotlin are all fine, as long as it properly supports OOP concepts such as classes, inheritance or interfaces, and encapsulation, since that is what gets assessed from Part D onward. At minimum, your code needs to support:

- Movie, cinema, screen, seat, and show representation
- Customer and booking management
- Show-specific seat availability (see Part C)
- Seat selection
- Simulated payment processing
- Booking confirmation
- Ticket generation
- A cancellation policy
- Simulated refund processing

Note: "Simulated" payment and refund do not require any real payment gateway. A simple success/failure flow, for example, a flag you can toggle or a basic condition you define, is enough. The focus of this lab is the object design and the process flow, not payment integration. Likewise, the cancellation policy is yours to define (for example, cancellations allowed only up to a certain time before the show). Just make sure it is applied consistently in your code and clearly explained in your Part A writeup.

9 Part F: Main / Test Driver and CLI Execution

Your classes define the data and behavior of the system. The Main / Test Driver is what brings it to life. Write an entry point that creates and connects the objects, exercises their methods, and runs through the required test cases. The program should be runnable from the command line. In Java that typically looks like `javac *.java` followed by `java Main`, other languages will have their own equivalent. Whatever you use, document the exact commands in your Run Instructions file so someone can run it without guessing.

Your CLI does not need to match this exactly, but it should support these operations:

1. View Movies
2. View Shows

3. View Available Seats
4. Book Tickets
5. View My Bookings
6. Cancel Booking
7. Run Required Test Cases
8. Exit

10 Part G: Required Test Cases

Run the test cases below using your Main / Test Driver. Fill in the actual result and PASS / FAIL status for each as you run it.

ID	Scenario	Expected Result
TC01	Successful booking	Available seat(s) are selected, payment succeeds, booking becomes CONFIRMED, and ticket(s) are generated.
TC02	Duplicate seat	A seat already booked for the same show is rejected.
TC03	Payment failure	The booking is not confirmed and no valid ticket is generated.
TC04	Cancellation with refund	An eligible cancellation changes the booking status, releases the seat, and generates the applicable refund.
TC05	Cancellation not allowed	The booking remains confirmed and no refund is issued.
TC06	Same seat, different shows	A seat booked for one show remains available for another show on the same screen.
TC07	Invalid seat	A non-existent seat number is rejected.

11 What to Submit

Your submission has five parts. Package all of them into a single zip file:

#	Deliverable	Format	What it should contain
1	Requirement Analysis & Design	PDF	Parts A, B, and C together: your identified classes with attributes, responsibilities, and justification; the relationships and multiplicities between them; and your reasoning on the seat-availability design question.

2	UML Class Diagram	PDF preferred (PNG is fine too)	The complete diagram from Part D. Export it as a PDF if your diagramming tool supports it (draw.io, Lucidchart, and Visio all do), since it stays sharp at any zoom level, which matters once a diagram has this many classes and relationships on it. A high-resolution PNG works as a fallback if a PDF export isn't convenient.
3	Source Code	Source files (any OOP language)	All class files from Part E plus your Main / Test Driver from Part F, in whatever language you implemented the system in. Keep them together in one folder so the project runs as-is. If your language needs a build or dependency file (a <code>requirements.txt</code> , a <code>pom.xml</code> , and so on), include that too.
4	Test Results	PDF or PNG screenshots	Output from running all seven required test cases (TC01 to TC07) with their PASS / FAIL status visible. If you're submitting screenshots, combine them into one PDF rather than leaving loose image files, it's much faster to review.
5	Run Instructions	.txt or PDF	A short set of steps to compile and run your program from the command line. A few lines is enough, it just needs to work if someone follows it exactly.

Zip everything into one file named `ID_Lab7.zip`, where `ID` is your student ID. Suggested layout:

```

ID_Lab7.zip
├── ID_Lab7_Report.pdf           (Parts A, B, C)
├── ID_Lab7_UML.pdf             (Part D diagram)
├── src/
│   ├── Main.<ext>
│   ├── Movie.<ext>
│   ├── Booking.<ext>
│   └── ...                     (rest of your class files, plus a build/dependency
│                               file if your language needs one)
├── ID_Lab7_TestResults.pdf     (Part G screenshots)
└── ID_Lab7_RunInstructions.txt

```

The point of this lab isn't just to get the code to compile. What matters is whether your UML design, your class implementation, and your Main / Test Driver actually show how the classes you identified collaborate to satisfy the requirement.